

Run Better Mocks and Measure Readiness

A mock is a diagnostic experiment. It should reveal how you clarify, reason, implement, test, communicate, and recover from feedback. Score these dimensions separately, then change one behavior and check it on a new task. A single pass or failure is not a reliable verdict.

For: Engineers who are studying regularly but cannot tell whether they are ready, or whose practice performance drops when a problem is timed and unfamiliar.

Suggested pace: A repeatable 4-week cycle | Candidates with basic preparation

What you will be able to do

- Design mocks that test transfer rather than recollection.
- Use a consistent rubric for coding, design, and project discussion.
- Turn vague feedback into one concrete practice intervention.
- Decide readiness using recent evidence without claiming a guaranteed interview outcome.

Build the right mental model

Keep the task unfamiliar

Choose an unseen problem or change the constraints of a known one. Hide the pattern category. If you already know the answer, use it as a rehearsal exercise and label it honestly; it should not count as independent diagnostic evidence.

Separate dimensions

A correct answer may still have weak testing or poor explanation. Score clarification, approach, correctness, complexity, implementation, tests, and communication on a simple scale. For design add requirement coverage, data flow, failure handling, and trade-offs.

Measure a trend

Use several sessions under similar conditions. Record hints and interruptions so comparisons are fair. Improvement means repeated correction of a weakness on different tasks, not merely a better score on the same memorized problem.

Study with evidence.

Use one or two focused mocks weekly, leaving time for correction. Explain the rubric to the reviewer beforehand. Afterward, select the highest-impact fix you can test during the next week rather than collecting a long list of vague criticisms.

Week 1: Establish a baseline

Run one coding mock and one role-relevant project or design discussion. Use the expected tools, but do not assume one universal employer timing. Record the problem, hints, completion point, and dimension scores.

Deliverable and check: A factual baseline with two recurring weaknesses and examples of where they appeared.

Week 2: Target the bottleneck

If recognition is weak, compare related patterns; if implementation is weak, rebuild small primitives; if testing is weak, practice adversarial examples. Keep the exercise narrow enough to see whether the intervention works.

Deliverable and check: A practice artifact that directly addresses a baseline failure, followed by an unseen transfer task.

Week 3: Increase realism

Run a mixed coding set and an architecture discussion with changing requirements. Ask the reviewer to interrupt with a legitimate constraint or failing test. Practice acknowledging uncertainty and revising the approach calmly.

Deliverable and check: A scorecard showing whether correctness and communication survive follow-up pressure.

Week 4: Decide and revise

Review recent sessions, not the best historical one. Keep strengths warm and revisit persistent gaps. Confirm the actual interview scope and logistics. Schedule additional preparation when evidence still shows fundamental instability.

Deliverable and check: A readiness decision with stated evidence, remaining gaps, and a final-week revision plan.

A repeatable practice session

Spend about 15 minutes recalling the previous topic without notes, 45 minutes on today's problem or lab, and 15 minutes explaining and checking the result. Record the failed assumption, not just the final answer. Revisit weak material after a few days and again the following week.

Apply it. Explain it.

Fix testing as an interview skill

Across three mocks, the candidate finds good algorithms but declares completion without testing. Two solutions contain boundary bugs. The correct intervention is not another advanced algorithm course.

A worked approach

1. Add a visible completion checklist: normal case, empty or minimal case when allowed, duplicates or ties, and a boundary condition specific to the invariant. Reserve a testing segment before the mock begins.
2. Practice tracing the smallest input that stresses each update rule. For binary search, check one element, missing target, and target at an endpoint; explain why the bounds still shrink and do not skip the answer.
3. Run a new mock and ask the reviewer to score testing separately. Compare whether the candidate identifies a bug before a hint and whether the final complexity statement matches the actual implementation.

Expected result

The intervention targets a repeated observable behavior. Success is demonstrated by better testing on a new task, not by adding a checklist to notes that is never used during execution.

Verify the result

- Are hints and independent discoveries recorded separately?
- Does the candidate explain why each test matters?
- Does the corrected behavior persist across more than one session?

Extend the exercise

Use the same rubric approach for design: require a write path, read path, and failure walkthrough before adding more components. Check whether this improves clarity on an unfamiliar system.

Make the learning stick.

What score means I am ready?

There is no employer-independent passing score. Define a personal gate using recent independent correctness, testing, communication, and role depth. The scorecard should support a decision, not imply a hiring probability you cannot measure.

Should a reviewer give hints?

Yes, when the mock format permits them, but record when and how much help was needed. Hint-assisted completion is useful learning evidence and should not be confused with an independent solve.

How do I handle harsh feedback?

Ask for a concrete example and an observable alternative behavior. Keep actionable technical evidence, clarify ambiguous criticism, and seek another perspective when feedback is inconsistent. One review is a data point, not a judgment of personal worth.

Avoid these traps

- Repeating familiar problems and counting them as unseen mocks.
- Judging only speed while ignoring correctness, tests, and clarity.
- Running frequent mocks without allocating time to fix the failures.

Ready for the next step?

- Recent mocks use unfamiliar tasks and recorded hints.
- Weak dimensions have specific, tested interventions.
- Explanations remain coherent under reasonable follow-up changes.
- A final revision plan is based on trends rather than one score.

Reliable next reads

- [Microsoft: problem solving, coding, and testing](#)
- [Amazon: coding and design preparation](#)

An independent preparation guide, not an employer-endorsed syllabus or a guarantee of an offer. Company examples are practice contexts, not verified past interview questions. Use only public, synthetic, or authorized material in exercises.